

Waste Image Classification

By Vineet

Here, we use an input image from outside the training dataset to check whether the model can classify a new image it has not seen during training.

For this example, we provide a photograph of magazines and observe whether the model correctly identifies the waste category as paper.

1 Project setup and input image

The supplied magazine photograph is saved as `inputs\mag.jpg`.



Expected material category by visual inspection: paper. The model result is recorded in section 6.

2 Prepare the dataset

Command: `python prepare_data.py`

This checks the existing waste images and saves the train, validation and test lists.

Training learns weights, validation selects the checkpoint, and testing measures the selected model.

Class	Train	Validation	Test
cardboard	282	60	61
glass	348	74	76
metal	286	61	62
paper	415	89	90
plastic	336	72	72
trash	95	20	22
Total	1762	376	383

Data preparation output

```
(.venv) PS C:\Users\vinee\OneDrive\Desktop\pytorch project> python prepare_data.py
Dataset folders already exist. Checking images next.
{
  "cardboard": {
    "train": 282,
    "val": 60,
    "test": 61
  },
  "glass": {
    "train": 348,
    "val": 74,
    "test": 76
  },
  "metal": {
    "train": 286,
    "val": 61,
    "test": 62
  },
  "paper": {
    "train": 415,
    "val": 89,
    "test": 90
  },
  "plastic": {
    "train": 336,
    "val": 72,
    "test": 72
  },
  "trash": {
    "train": 95,
    "val": 20,
    "test": 22
  }
}
Removed 0 same-class duplicates and excluded 3 conflicting groups.
```

3 Check the project

Command: `python check_project.py`

```
saved C:\Users\vinee\OneDrive\Desktop\pytorch project\outputs\split.json  
(.venv) PS C:\Users\vinee\OneDrive\Desktop\pytorch project> python .\check_project.py  
PASS: splits are disjoint and contain every class  
PASS: RGB tensor shapes, types and normalization  
PASS: output dimensions, finite gradients and parameter updates  
PASS: repeatable evaluation without gradients; probabilities sum to one  
Model parameters: 155,558
```

The checks use a separate temporary model and do not change the saved classifier.

15,5558 parameters means the model contains that many trainable weights and biases.

PASS confirms these code checks succeeded; it does not mean every image will be classified correctly.

4 Train the Model

Command: `python train.py --epochs 20`

This command trains a fresh model and replaces the saved checkpoint. The table records the completed run. The best checkpoint is selected using the lowest validation loss, not the last epoch.

Epoch	Train loss	Train accuracy	Val loss	Val accuracy
1	1.5835	33.20%	1.4298	36.44%
2	1.4000	41.26%	1.3062	47.07%
3	1.2580	48.41%	1.2052	54.26%
4	1.1468	55.96%	1.2226	51.33%
5	1.0899	59.31%	1.0847	59.04%
6	1.0197	61.24%	1.1321	57.98%
7	1.0085	61.92%	1.1040	60.64%
8	0.9638	64.36%	1.0504	58.78%

Epoch	Train loss	Train accuracy	Val loss	Val accuracy
9	0.8853	66.74%	0.9814	62.50%
10	0.8458	69.98%	0.9871	61.97%
11	0.8067	70.43%	0.9225	63.30%
12	0.7910	71.17%	0.8882	65.96%
13	0.7287	72.53%	0.9490	64.63%
14	0.7362	72.30%	0.9144	65.96%
15	0.6801	74.97%	0.8334	70.74%
16	0.6870	74.63%	0.8764	67.02%
17	0.6433	76.22%	0.8960	67.82%
18	0.5860	78.04%	0.8546	68.09%
19	0.5868	77.70%	0.8604	71.01%
20	0.5332	80.02%	0.8666	68.35%

Selected checkpoint: epoch 15. Validation loss: 0.8334. Validation accuracy: 70.74%.

```
(.venv) PS C:\Users\vinee\OneDrive\Desktop\pytorch project> python .\train.py --epochs 20
Training on cpu. Classes: ['cardboard', 'glass', 'metal', 'paper', 'plastic', 'trash']
Epoch 01/20: train loss=1.5835, accuracy=33.20%; val loss=1.4298, accuracy=36.44%
Epoch 02/20: train loss=1.4000, accuracy=41.26%; val loss=1.3062, accuracy=47.07%
Epoch 03/20: train loss=1.2580, accuracy=48.41%; val loss=1.2052, accuracy=54.26%
Epoch 04/20: train loss=1.1468, accuracy=55.96%; val loss=1.2226, accuracy=51.33%
Epoch 05/20: train loss=1.0899, accuracy=59.31%; val loss=1.0847, accuracy=59.04%
Epoch 06/20: train loss=1.0197, accuracy=61.24%; val loss=1.1321, accuracy=57.98%
Epoch 07/20: train loss=1.0085, accuracy=61.92%; val loss=1.1040, accuracy=60.64%
Epoch 08/20: train loss=0.9638, accuracy=64.36%; val loss=1.0504, accuracy=58.78%
Epoch 09/20: train loss=0.8853, accuracy=66.74%; val loss=0.9814, accuracy=62.50%
Epoch 10/20: train loss=0.8458, accuracy=69.98%; val loss=0.9871, accuracy=61.97%
Epoch 11/20: train loss=0.8067, accuracy=70.43%; val loss=0.9225, accuracy=63.30%
Epoch 12/20: train loss=0.7910, accuracy=71.17%; val loss=0.8882, accuracy=65.96%
Epoch 13/20: train loss=0.7287, accuracy=72.53%; val loss=0.9490, accuracy=64.63%
Epoch 14/20: train loss=0.7362, accuracy=72.30%; val loss=0.9144, accuracy=65.96%
Epoch 15/20: train loss=0.6801, accuracy=74.97%; val loss=0.8334, accuracy=70.74%
Epoch 16/20: train loss=0.6870, accuracy=74.63%; val loss=0.8764, accuracy=67.02%
Epoch 17/20: train loss=0.6433, accuracy=76.22%; val loss=0.8960, accuracy=67.82%
Epoch 18/20: train loss=0.5860, accuracy=78.04%; val loss=0.8546, accuracy=68.09%
Epoch 19/20: train loss=0.5868, accuracy=77.70%; val loss=0.8604, accuracy=71.01%
Epoch 20/20: train loss=0.5332, accuracy=80.02%; val loss=0.8666, accuracy=68.35%
Best validation model saved to C:\Users\vinee\OneDrive\Desktop\pytorch project\outputs\best_model.pth
```

5 Evaluate the saved model

Command: `python evaluate.py`

Evaluation loads the saved model and uses 383 held out test images. The two charts go to outputs.

Measure	Recorded result
Test accuracy	70.23% or 269 of 383 correct
Test loss	0.8251
F1	0.6804
Always predict paper baseline	23.50%
Selected epoch	15

Per class results from this evaluation

Class	Precision	Recall	F1	Images
cardboard	0.800	0.787	0.793	61
glass	0.705	0.724	0.714	76
metal	0.686	0.565	0.619	62
paper	0.773	0.756	0.764	90
plastic	0.614	0.708	0.658	72
trash	0.522	0.545	0.533	22

Precision measures how often a predicted class is correct.

Recall measures how many actual images of that class are found.

F1 combines them.

```

• (.venv) PS C:\Users\vinee\OneDrive\Desktop\pytorch project> python .\evaluate.py
{
  "test_images": 383,
  "test_loss": 0.8251336202920261,
  "test_accuracy": 0.7023498694516971,
  "macro_f1": 0.6804309939781739,
  "majority_baseline_class": "paper",
  "majority_baseline_accuracy": 0.2349869451697128,
  "selected_epoch": 15,
  "validation_accuracy": 0.7074468085106383,
  "per_class": {
    "cardboard": {
      "precision": 0.8,
      "recall": 0.7868852459016393,
      "f1": 0.7933884297520661,
      "support": 61
    },
    "glass": {
      "precision": 0.7051282051282052,
      "recall": 0.7236842105263158,
      "f1": 0.7142857142857143,
      "support": 76
    },
    "metal": {
      "precision": 0.6862745098039216,
      "recall": 0.5645161290322581,
      "f1": 0.6194690265486725,
      "support": 62
    },
    "paper": {
      "precision": 0.7727272727272727,
      "recall": 0.7555555555555555,
      "f1": 0.7640449438202247,
      "support": 90
    },
  },
}

```

```

    },
    "plastic": {
      "precision": 0.6144578313253012,
      "recall": 0.7083333333333334,
      "f1": 0.6580645161290323,
      "support": 72
    },
    },
    "trash": {
      "precision": 0.5217391304347826,
      "recall": 0.5454545454545454,
      "f1": 0.5333333333333332,
      "support": 22
    }
  }
}

```

6 Predict the supplied magazine image

Command: `python predict.py "inputs\mag.jpg"`

Observed predicted class: **paper**. The image shows magazines, so the prediction agrees with the visually expected material category.

Category	Softmax score
paper	0.9990185499
cardboard	0.0005799625
metal	0.0003138194
glass	0.0000769081
plastic	0.0000106904
trash	0.0000000007

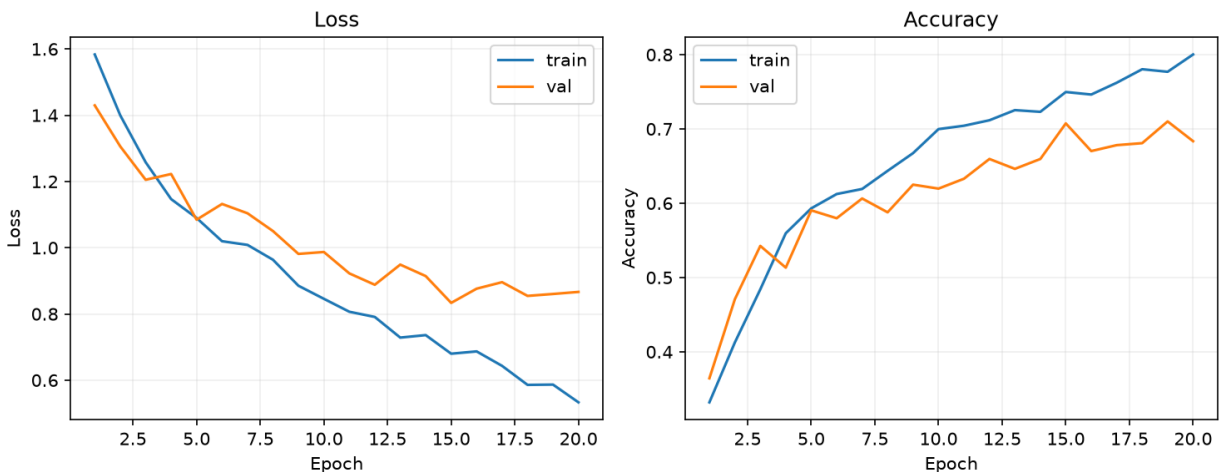
The highest score is 99.90% for paper. Scores sum to approximately one. This is not the overall model accuracy or guaranteed certainty. The classifier always chooses one of its six known labels.

Magazine prediction output

```
(.venv) PS C:\Users\vinee\OneDrive\Desktop\pytorch project> python predict.py "C:\Users\vinee\Downloads\mag.jpg"
{
  "image": "C:\\Users\\vinee\\Downloads\\mag.jpg",
  "predicted_class": "paper",
  "softmax_scores": {
    "paper": 0.9990185499191284,
    "cardboard": 0.0005799625068902969,
    "metal": 0.00031381938606500626,
    "glass": 7.690812344662845e-05,
    "plastic": 1.0690366252674721e-05,
    "trash": 6.713282707515589e-10
  }
}
```

7 Training curves

start .\outputs\training_curves.png



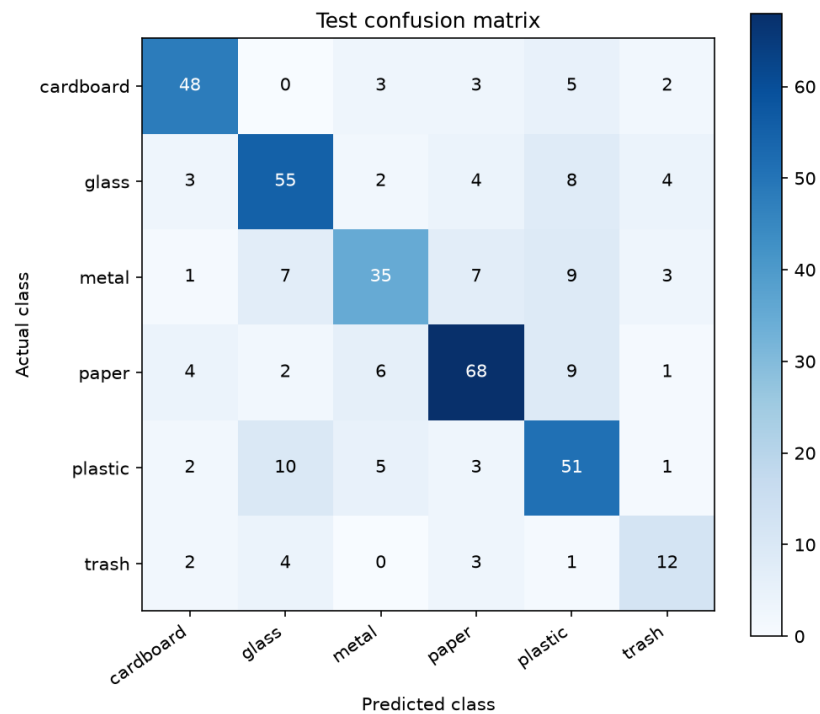
Left chart: lower loss is better.

Right chart: higher accuracy is better.

Blue is training and orange is validation. Training improves across epochs, while validation loss is lowest at epoch 15. Later divergence suggests some overfitting. The chart summarizes training on the dataset.

8 Confusion matrix

start .\outputs\confusion_matrix.png



Rows are actual classes and columns are predicted classes. Diagonal values are correct predictions. For example, 48 of 61 cardboard images were classified correctly; 5 were mistaken for plastic. Across the diagonal, 269 images were correct.